

TagFex: Task-Agnostic Guided Feature Expansion for Class-Incremental Learning Report

Umanshiva Ladva (AI22BTECH11016)
Teaching Assistant: Rahul Biswas

Abstract

This report presents a thorough study of TagFex: Task-Agnostic Guided Feature Expansion, a recent framework for class-incremental learning (CIL). The study involves (i) understanding the original methodology, (ii) replicating the reported results, (iii) analyzing its limitations, and (iv) proposing and experimenting with a novel extension using prompt-based continual learning (L2P). The experiments cover CIFAR-100 10-10 settings, comparing original results with my own implementation. Further, I analyze representational interactions, prompt placement issues, and architectural incompatibilities that emerged during experimentation. The study concludes with potential improvements, future directions, and insights acquired from replication.

1. Introduction

Continual learning (CL) aims to sequentially learn new tasks without catastrophic forgetting. A challenging variant is **Class-Incremental Learning (CIL)**, where the model must classify across both old and new classes without task identifiers at test time. Traditional CL methods often suffer from issues such as:

- **Catastrophic Forgetting:** new tasks overwrite old knowledge.
- **Class Imbalance:** older tasks contribute fewer samples.
- **Stability–Plasticity trade-off:** Retaining and preserving the knowledge from old tasks vs Learning new tasks

Recent methods attempt to address these challenges using model expansion, rehearsal, or regularization. TagFex[5] differentiates itself by leveraging **task-agnostic feature learning** from self-supervised learning and integrating them with task-specific features.

This report explores the TagFex framework in depth and evaluates its effectiveness through implementation and experimentation.

2. Related Work

2.1. Class-Incremental Learning

Early CIL methods relied heavily on regularization, such as EWC and LwF, but these struggled as the number of tasks increased. Rehearsal-based methods store a small set of samples from previous tasks, improving stability but requiring memory. Expansion-based models such as DER [4] dynamically add new modules per task, offering strong performance but increasing model size.

2.2. Self-Supervised Continual Learning

CSSL methods like CaSSLe [1] demonstrate that models trained through contrastive learning can maintain stable representations without catastrophic forgetting. These works inspire architectures such as TagFex that integrate SSL with incremental learning.

2.3. Prompt-Based Continual Learning

Learning to Prompt[3] (L2P) introduces a training-free and task-agnostic prompting mechanism for CL. Instead of expanding the model or replaying old data, L2P uses a pool of prompt vectors that adapt the representation for each new task. This reduces computation and memory demands but requires careful architectural alignment.

3. Problem Statement

In CIL scenarios, we have multiple classification tasks to learn sequentially. Let D_t be the training dataset of the t th task. $(x_i^{(t)}, y_i^{(t)}) \in D_t$ is a sample. $x_i^{(t)}$ is the input, $y_i^{(t)}$ is the label. Let $C_t = \bigcup_i \{y_i^{(t)}\}$ be the class set of task t . In CIL, $\forall t_1 \neq t_2, C_{t_1} \cap C_{t_2} = \phi$. In each task, we only train the model on D_t , but test on all the tasks the model has trained on, i.e., the seen tasks. For example, when the model is training on task t_i , the seen tasks are tasks $t_j (j \leq i)$. Therefore, the class set of the seen tasks is $Y_t = \bigcup_{t'} C_{t'}$. In rehearsal-based CIL, a small number of rehearsal samples M are allowed to be stored for later tasks. The number of

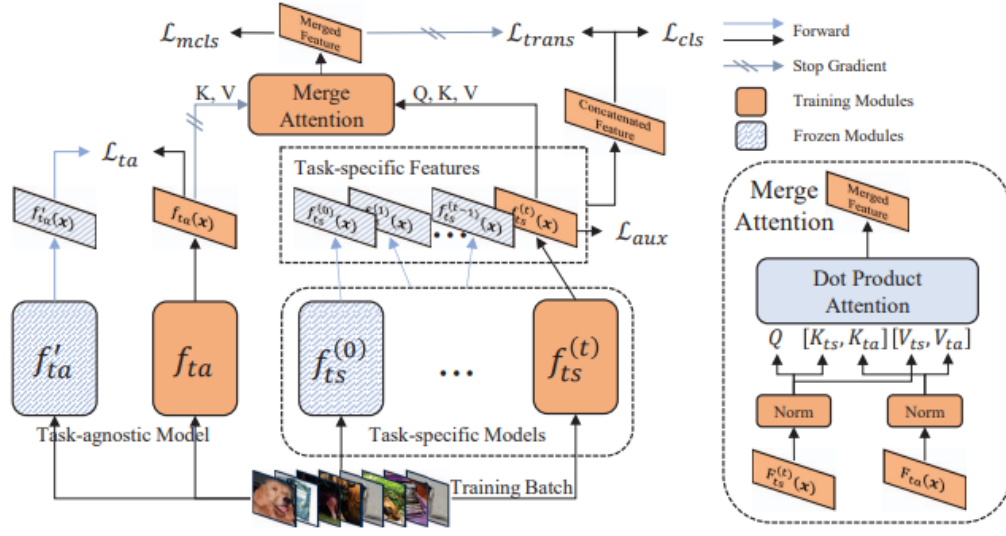


Figure 1. TagFex Architecture

rehearsal samples should be constant during the CIL training. The goal is to make the model get better performance on all the seen tasks.

4. TagFex: Background and Methodology

4.1. Motivation

A supervised model trained on an incremental task typically captures only the discriminative features necessary for that task. This results in **task-specific features** that may not generalize across tasks. However, there exist additional latent features in the data that can be learned in a task-agnostic manner. TagFex exploits this insight by splitting the learning process into two parallel streams:

1. **Task-Specific Features (TS)**: learned via supervised training.
2. **Task-Agnostic Features (TA)**: learned via continual self-supervised learning.

The TA model is inspired by CaSSLe, which maintains predictive features across tasks by enforcing consistency through augmentations and temporal models.

4.2. Preliminaries: DER and Task-Specific Feature Expansion

Dynamically Expandable Representation (DER)[4] serves as the foundation for the task-specific feature expansion strategy used in our work. In DER, a new task-specific model $f_{ts}^{(t)}$ is instantiated at the beginning of each incremental task, while all previously trained models $f_{ts}^{(0)}, \dots, f_{ts}^{(t-1)}$ are frozen and preserved for feature extraction. The classifier is expanded correspondingly, inheriting previously learned parameters. During inference, the con-

catenated features from all task-specific models are fed into the classifier, and the resulting classification objective for a sample (x, y) is computed via standard cross-entropy.

$$L_{cls}(\mathbf{x}, y) = l_{CE}([f_{ts}^{(0)}(\mathbf{x}), f_{ts}^{(1)}(\mathbf{x}), \dots, f_{ts}^{(t)}(\mathbf{x})], y; W_{cls}^{(t)}) \quad (1)$$

To encourage the learning of more diverse features within each task, DER additionally employs an auxiliary classifier that operates on the current task-specific features. Unlike the main classifier, this auxiliary classifier must distinguish both the new classes and the samples belonging to any earlier task, which is achieved by maintaining a small rehearsal buffer. Consequently, it outputs $|C_t| + 1$ logits and is trained using a cross-entropy objective.

$$L_{aux}(\mathbf{x}, \mathbf{y}) = l_{CE}(f_{ts}^{(t)}(\mathbf{x}), \mathbf{y}; W_{aux}^{(t)}) \quad (2)$$

Under supervised learning for a single task, a model typically captures only *task-specific features*—the minimal discriminative cues required for that task. Such features do not generalize across tasks. However, real-world data contains richer structure beyond these minimal cues, leading to the notion of *task-agnostic features*, which are not necessarily tied to any specific task but are broadly useful for representation learning. DER partially enhances feature diversity through rehearsal and auxiliary classification, but its capability remains constrained by the limited buffer size and class imbalance. These limitations motivate the incorporation of task-agnostic features into the task-specific learning pipeline, which forms the basis for the improvements explored in the subsequent sections.

4.3. Framework Overview

TagFex is composed of three components:

- **Continual Acquisition of TA Features:** a separate SSL branch trained using InfoNCE-based contrastive learning.
- **Adaptive Feature Aggregation via Merge Attention:** combines TA and TS features via cross-attention to selectively absorb useful task-agnostic knowledge.
- **Task-Agnostic Guided Knowledge Transfer:** KL divergence used to distill merged knowledge back into the task-specific classifier.

The overall loss is:

$$L = \lambda_{ta}L_{ta} + \lambda_{mcls}L_{mcls} + \lambda_{ts}L_{ts}$$

where L_{ts} includes classification, auxiliary discrimination, and transfer losses.

4.4. Why TagFex Works

TagFex improves existing expansion-based methods (e.g., DER) by enabling each task-specific model to learn richer feature sets guided by preserved task-agnostic knowledge. The adaptive merging allows task-specific models to leverage previous predictive representations without relying exclusively on a limited rehearsal buffer.

5. Replication of TagFex Results

5.1. Experimental Setup

Experiments were conducted in the CIFAR-100 10-10 class-incremental setting. Hyperparameters differed from the original paper due to time and compute constraints (Table 1).

Setting	My Value	Original
Base Epochs	70	200
Incremental Epochs	50	170
Batch Size	128	128
Learning Rate	0.1	0.1

Table 1. Hyperparameter Comparison

Despite reduced training time, obtained results reasonably align with expected behaviours.

5.2. Results

Scenario	Last	Average
Original Paper	68.23	78.45
My Replication (Acc-1)	66.13	74.00
My Replication (Acc-5)	88.99	93.56

Table 2. Replicated vs. Reported Results

5.3. Limitations

There are a few but important limitations to the proposed method:

- Expansion modules add extra parameters which leads to higher memory cost than pure regularization
- Requires careful balancing of expansion vs. overfitting
- Relies on task-dependent expansions for feature extraction across tasks. This leads to reduced scalability for real-world Continual Learning settings.

6. Prompt-Based Extension: L2P Integration

6.1. Motivation

TagFex relies heavily on expansion-based per-task modules. This leads to:

- Linear increase in parameters per task.
- Higher GPU memory consumption.
- Reduced scalability for many-task scenarios.

To address these limitations, I explored replacing expansion modules with **Learning to Prompt (L2P)**, a prompt-tuning based CL approach.

6.2. Understanding L2P

L2P uses a pool of learnable prompts. During training and inference:

1. image features are extracted via a pretrained backbone;
2. similarity-based retrieval selects relevant prompts;
3. prompts are prepended to the token sequence (or feature tensor);
4. the model adapts to the new task without expanding architecture.

This approach benefits in the following ways:

- Scalable and task-agnostic
- No need for rehearsal or task identifiers
- Lightweight in terms of parameter growth

6.3. Integration Attempt

The idea was to replace TagFex’s per-task feature extractors with:

- A fixed pre-trained ResNet-18 backbone(original L2P had ViT as backbone. Due to computational resource limitation I used this alternative)
- A trainable prompt pool

The resulting architecture inserts prompts into the *task-specific module* before merging with TA features.

The benefits of this approach are:

- Task-specific Module remains fixed-size across tasks.
- Prompts specialize naturally for each task

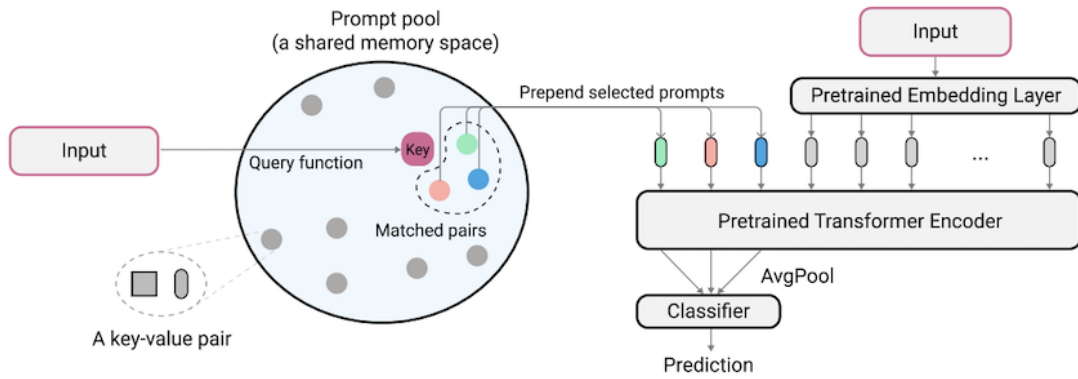


Figure 2. L2P

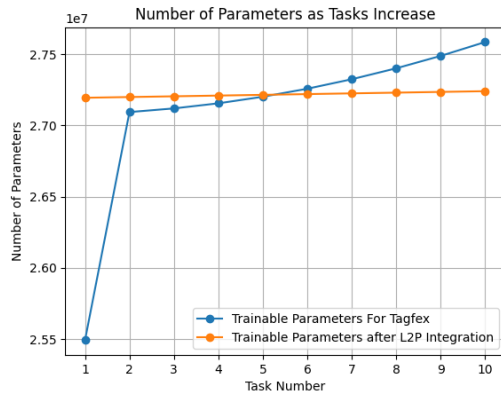


Figure 3. Parameter Growth Comparison

- Compatible with TagFex’s modular learning pipeline. Meaning that, since each part is isolated and independent module, we can swap one module with another

6.4. Observed Results

Scenario	Last	Average
Accuracy-1	6.47	24.26
Accuracy-5	9.72	28.85

Table 3. Performance of L2P + TagFex Integration

Performance dropped significantly.

6.5. Analysis: Why Did Performance Drop?

Few of the plausible reasons that might be the cause of the performance drop:

1. **Prompt Insertion Location:** Prompts inserted before merging likely disrupted:

- task-specific feature structure,
- compatibility with TA features,
- merge-attention behaviours.

2. **Reduced Representational Capacity:** TagFex relies strongly on diversity from per-task expansions. Replacing them with a fixed backbone + small prompt pool reduces flexibility.
3. **Architectural Mismatch:** TagFex assumes TS features come from a learnable model; prompts produce perturbations inconsistent with that assumption.

7. Possible Improvements and Future Directions

- Insert prompts into the **merged feature space** rather than TS space to reduce the interference in the shared feature space among the tasks.
- Increase prompt pool size for better specialization. This might lead to less task sharing
- Can shift to more robust prompt based CL approaches such as **DualPrompt** [2].
- Use pretrained ViT backbones for better representation capacity and compatibility with prompting.
- Train for more epochs per task to better differentiate task-specific and task-agnostic features.

8. Conclusion

This study successfully explored TagFex from both a theoretical and practical perspective. By replicating results, analyzing training dynamics, and proposing a novel prompt-based modification, deeper insights were gained into:

- modular CL architectures,
- feature space interactions,
- merging task-agnostic and task-specific representations,
- stability and representational bottlenecks,

- architectural compatibility for plug-in modules.

Although the prompt-based extension underperformed, it highlights important lessons about how continual learning frameworks must be carefully designed for feature-space compatibility.

Overall, this work deepened my understanding of state-of-the-art Class Incremental Learning techniques and revealed promising directions for future research toward efficient, scalable, and modular continual learning systems.

References

- [1] Enrico Fini, Victor G. Turrissi da Costa, Xavier Alameda-Pineda, Elisa Ricci, Karteek Alahari, and Julien Mairal. Self-supervised models are continual learners, 2022.
- [2] Zifeng Wang, Zizhao Zhang, Sayna Ebrahimi, Ruoxi Sun, Han Zhang, Chen-Yu Lee, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, and Tomas Pfister. Dualprompt: Complementary prompting for rehearsal-free continual learning, 2022.
- [3] Zifeng Wang, Zizhao Zhang, Chen-Yu Lee, Han Zhang, Ruoxi Sun, Xiaoqi Ren, Guolong Su, Vincent Perot, Jennifer Dy, and Tomas Pfister. Learning to prompt for continual learning, 2022.
- [4] Shipeng Yan, Jiangwei Xie, and Xuming He. Der: Dynamically expandable representation for class incremental learning, 2021.
- [5] Bowen Zheng, Da-Wei Zhou, Han-Jia Ye, and De-Chuan Zhan. Task-agnostic guided feature expansion for class-incremental learning, 2025.